

RELATÓRIO PROJETO PRÁTICO



Professora: Marluce Rodrigues Pereira

Aluno 1: Enzo Ribeiro Almeida - **Matrícula Nº:** 202611181 - **TURMA 10B**

Aluno 2: João Pedro Ribeiro Santiago - **Matrícula Nº:** 202610128 - **TURMA 10B**

Aluno 3: João Vitor Melo Alves Freitas - **Matrícula Nº:** 202610130 - **TURMA 10B**

1. Introdução Inicial –

A ideia do projeto foi colocar em prática tudo o que aprendemos nesse primeiro período. Em vez de usar as funções prontas da linguagem, nós implementamos toda a lógica. O sistema lê os dados de um arquivo e guarda tudo na memória, usando alocação dinâmica, o que significa que o nosso programa é inteligente o suficiente para perceber quando o espaço está acabando e aumentar o tamanho da lista sozinho. Nós também programamos nossos próprios algoritmos para organizar e procurar os personagens. Dependendo do que o usuário quer fazer, o sistema decide se vai usar métodos de ordenação como o Selection Sort, e se vai caçar a informação usando uma Busca Binária ou uma Busca Sequencial. No final, todas as mudanças, como adicionar ou excluir um campeão, podem ser salvas de volta no arquivo original para não perdermos nada.

2. Descrição em alto nível –

O programa funciona como um arquivo digital inteligente que carrega, organiza, modifica e salva informações de forma contínua e adaptável. A lógica central opera inteiramente na memória do computador, o que garante rapidez durante o uso.

2.1 - Estrutura de Armazenamento e Flexibilidade

- **Fichas de Dados:** Cada campeão é tratado como uma "ficha" única que agrupa características distintas: textos (nome, classe, região, ano de lançamento) e números (a taxa de escolha, ou pick rate).
- **Lista Elástica:** Todas essas fichas são enfileiradas em uma lista na memória que consegue se expandir e encolher. Se a lista lotar e um novo campeão precisar entrar, o programa constrói uma lista maior nos bastidores, transfere todo mundo para a nova casa e destrói a antiga para não desperdiçar a memória. Se um campeão é removido, a lista encolhe proporcionalmente.

2.2 - Inteligência de Busca

Para encontrar informações rapidamente em uma lista que pode crescer muito, o sistema utiliza duas estratégias principais:

- **A Abordagem do Dicionário (Busca Binária):** Quando o usuário procura um campeão pelo nome, o programa não olha ficha por ficha. Ele vai direto ao meio da lista e verifica se o nome procurado vem antes ou depois dali. Assim, ele descarta metade da lista instantaneamente e repete o processo até achar o alvo. Para isso funcionar, o programa garante que a lista esteja sempre em rigorosa ordem alfabética.
- **A Abordagem do Censo (Busca Sequencial):** Quando a busca é por uma "região" (onde vários campeões podem morar), a estratégia do dicionário não serve. Nesse caso, o sistema percorre cada ficha, da primeira à última, separando e agrupando todos os personagens que pertencem àquele local.

2.3 - Algoritmos de Organização

Sempre que um novo dado entra ou sai, a lista corre o risco de ficar bagunçada. Para evitar isso, o programa aplica métodos matemáticos de organização. Ele varre a lista comparando as fichas entre si e trocando-as de posição até que tudo fique perfeitamente alinhado. O sistema permite que o usuário escolha o critério dessa arrumação: pode ser por ordem alfabética ou cronológica (ano de lançamento).

2.4 - Persistência de Dados

Todo o trabalho de busca, adição, remoção e ordenação acontece em um ambiente temporário (a memória do computador). Para que o trabalho não seja perdido ao desligar o sistema, existe uma lógica de salvamento que pega o estado atualizado dessa lista elástica e sobrescreve o arquivo de texto original no disco rígido, consolidando todas as mudanças.

3. Descrição da ordem dos dados armazenados no arquivo –

Os dados no arquivo do programa são armazenados de forma estritamente sequencial, divididos entre um cabeçalho de controle inicial e a lista contínua de personagens. As quatro primeiras linhas funcionam como metadados essenciais para o sistema saber como remontar o vetor na memória ao ser iniciado. A primeira linha contém apenas um texto de comentário com o nome das colunas, que é lido e descartado logo em seguida. A segunda linha guarda um número inteiro que representa a capacidade máxima atual do vetor, enquanto a terceira linha armazena o tamanho real da lista, ou seja, a quantidade exata de campeões cadastrados. A quarta linha, por sua vez, indica por um número a ordem em que o arquivo foi salvo, como alfabética ou cronológica. A partir da quinta linha, o arquivo passa a listar o banco de dados em si, onde cada linha inteira corresponde a um campeão único. Para que a leitura do sistema não quebre, as informações do personagem seguem uma sequência fixa e obrigatória: primeiro vem o nome, seguido pela classe, depois a região, o ano de lançamento e, por último, o número decimal do pickrate. Todos esses atributos são separados por vírgulas, e o registro de cada campeão é sempre finalizado com um ponto e vírgula, garantindo a organização exata que o código espera encontrar.

3.1 - Exemplo no Arquivo “LOL.csv” –

```
#NOME,CLASSE,REGIAO,ANO DE LANÇAMENTO,PICKRATE EM %;  
40  
172  
1  
AATROX,LUTADOR,RUNETERRA,2013,6.3;  
AHRI,MAGO,IONIA,2011,10.46;
```

4. Acertos e erros durante o desenvolvimento do trabalho –

4.1 Erros e Dificuldades Enfrentadas

- **Manipulação de Arquivos (getline):** Houve uma dificuldade inicial com a lógica de extração de dados do CSV. A sintaxe e o comportamento do getline geraram conflitos com a formatação original do arquivo, exigindo que a estrutura do documento fosse alterada e simplificada para que a importação funcionasse sem quebrar o código.
- **Gerenciamento de Memória e Ponteiros:** O desenvolvimento das funções de adicionar e remover campeões esbarrou em erros críticos de acesso à memória (segmentation fault). Isso demonstrou um desafio inicial natural em manipular vetores dinâmicos, realocar tamanhos e reposicionar elementos (fazer o shift dos dados) sem acessar posições inválidas.

- **Conflito entre Ordenação e Busca:** A tentativa de realizar pesquisas em um vetor que estava ordenado cronologicamente (por ano) gerou inconsistências. A busca perdia eficiência e precisão, pois os algoritmos de pesquisa rápida (como a Busca Binária) exigem que os dados estejam ordenados pelo mesmo critério que está sendo buscado.

4.2 Acertos e Soluções Implementadas

- **Tratamento e Padronização de Entradas:** A criação do procedimento para converter as strings para letras maiúsculas foi uma excelente decisão de design. Isso resolveu o problema do case sensitivity (diferença entre maiúsculas e minúsculas), tornando as buscas robustas e evitando frustrações do usuário na hora de digitar o nome do personagem.
- **Resolução de Falhas Críticas:** Superar os segmentation faults e conseguir estabilizar a alocação dinâmica para inclusão e remoção de dados mostra que a equipe conseguiu absorver a lógica de ponteiros (new e delete) e aplicá-la com sucesso de forma funcional.
- **Adaptabilidade Prática:** A percepção de que a busca no vetor ordenado por ano estava ineficiente resultou em uma adaptação inteligente na lógica do código: priorizar a ordenação por nome no momento de realizar buscas específicas. Isso demonstra flexibilidade da equipe para contornar problemas de design e priorizar o funcionamento e a usabilidade do sistema.

5. Conclusão Final –

A conclusão do trabalho demonstra que o objetivo principal do projeto foi alcançado com sucesso, resultando em um sistema de gerenciamento de dados totalmente funcional e estável, capaz de ler, armazenar, manipular e salvar informações dos campeões de League of Legends de forma persistente e dinâmica. Apesar das dificuldades técnicas como os erros de memória durante a alocação de ponteiros e os obstáculos com a leitura do arquivo CSV, as soluções desenvolvidas pela equipe garantiram a robustez do programa, destacando-se a padronização das entradas de texto para letras maiúsculas, que eliminou falhas de digitação durante as buscas, e a adaptação da ordenação do vetor antes de pesquisas específicas para otimizar a precisão dos algoritmos. Como resultado prático e acadêmico, o sistema atende a todos os requisitos propostos sem depender de funções prontas da linguagem para a organização dos dados, rodando sem travamentos, atualizando o banco de dados corretamente e provando que a equipe conseguiu converter a teoria de algoritmos, estruturas dinâmicas e manipulação de arquivos em um software eficiente e operante.